Eternally Yours - Technical Implementation

Architecture & Tech Stack

The platform is built using a modern, decoupled architecture ensuring high performance, scalability, and maintainability.

- **Frontend:** Angular (Standalone Components, RxJS, TypeScript)
- **Backend:** Django & Django REST Framework (DRF)
- **Real-time Communication:** Django Channels & WebSockets
- **Database:** Relational Database (via Django ORM)
- **Data Visualization:** Chart.js (ng2-charts)
- **Mapping:** Leaflet.js

Backend Implementation

- **RESTful APIs:** Built with DRF, handling authentication, event retrieval, vendor management, and bookings.
- **Real-time Chat:** Django Channels routes WebSocket connections to allow seamless, instant messaging between users and event organizers. Messages are persisted in the `ChatMessage` model.
- **AI Chatbot Integration:** The `AIChatView` processes natural language queries, dynamically searches the `Event` database using keyword matching and categorization, and returns structured vendor data including images and location links.
- **Data Models:**
    - `User`: Custom user model managing roles (Admin, Client/Organizer, User).
    - `Event` & `Venue`: Core models storing service details, pricing, gallery images, and geographic coordinates.
    - `Enquiry`: Tracks lead generation whenever a user initiates contact with a vendor.
    - `Subscription`: Manages vendor access levels and approval statuses.

Frontend Implementation

- **Design System:** A robust, CSS-variable driven theme system supporting Dark Mode and multiple color palettes (Amethyst, Gold, etc.). Employs glassmorphism and fluid animations for a premium feel.
- **Component Structure:**
    - `AdminDashboard`: Features a custom chart designer, tabbed data tables, and high-level platform controls.
    - `EventDetail`: Integrates Leaflet maps, a dynamic photo gallery lightbox, and the live WebSocket chat interface.
    - `Chatbot`: A globally accessible floating widget that renders dynamic "Mini Vendor Cards" directly in the chat stream based on API responses.
- **State Management & Services:**
    - `AuthService` manages JWT tokens, user roles, and theme preferences.
    - `EventService` and `InteractionService` handle HTTP requests and WebSocket lifecycle management.

Security & Performance

- **Authentication:** Token-based authentication securing API endpoints and routing guards protecting frontend views.
- **Role-Based Access Control (RBAC):** Strict segregation between Admin, Client (Vendor), and regular User permissions.
- **Optimized Assets:** Lazy loading of routes, optimized CSS rendering, and responsive image handling.